



Reducing Java Cold Starts with AWS Lambda SnapStart

Performance Optimization of Java 21 Serverless Applications

N RAVINDRA 2400030019

M V SAI KALYAN 2400030027

KADA KOUSHIK 2400030046

JAMI JASWANTH 2400030079

BATCH NUMBER: 117

GUIDED BY:
SUNEETHA BANDEELA

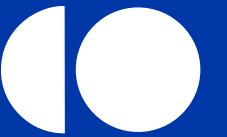


Agenda

1. Introduction to AWS Lambda and Cold Starts
2. SnapStart Overview and Experiment Setup
3. Results, Benefits, and Next Steps



Introduction to AWS Lambda and Cold Starts



What AWS Lambda Does



Serverless execution model

- AWS Lambda runs your code without provisioning or managing servers.
- It automatically scales capacity up or down with incoming demand.



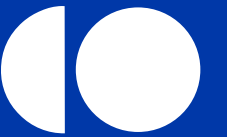
Event-driven by design

- Functions can be triggered by APIs, file uploads, database events, or schedules.
- You focus on application logic while AWS handles the infrastructure work.



Why startup speed matters

- When a function starts fresh, users can feel the extra startup delay.
- Reducing cold-start time improves perceived responsiveness for quick requests.



Why Cold Starts Happen

A new environment is created

A cold start happens when Lambda must spin up a fresh runtime.

Startup has multiple stages

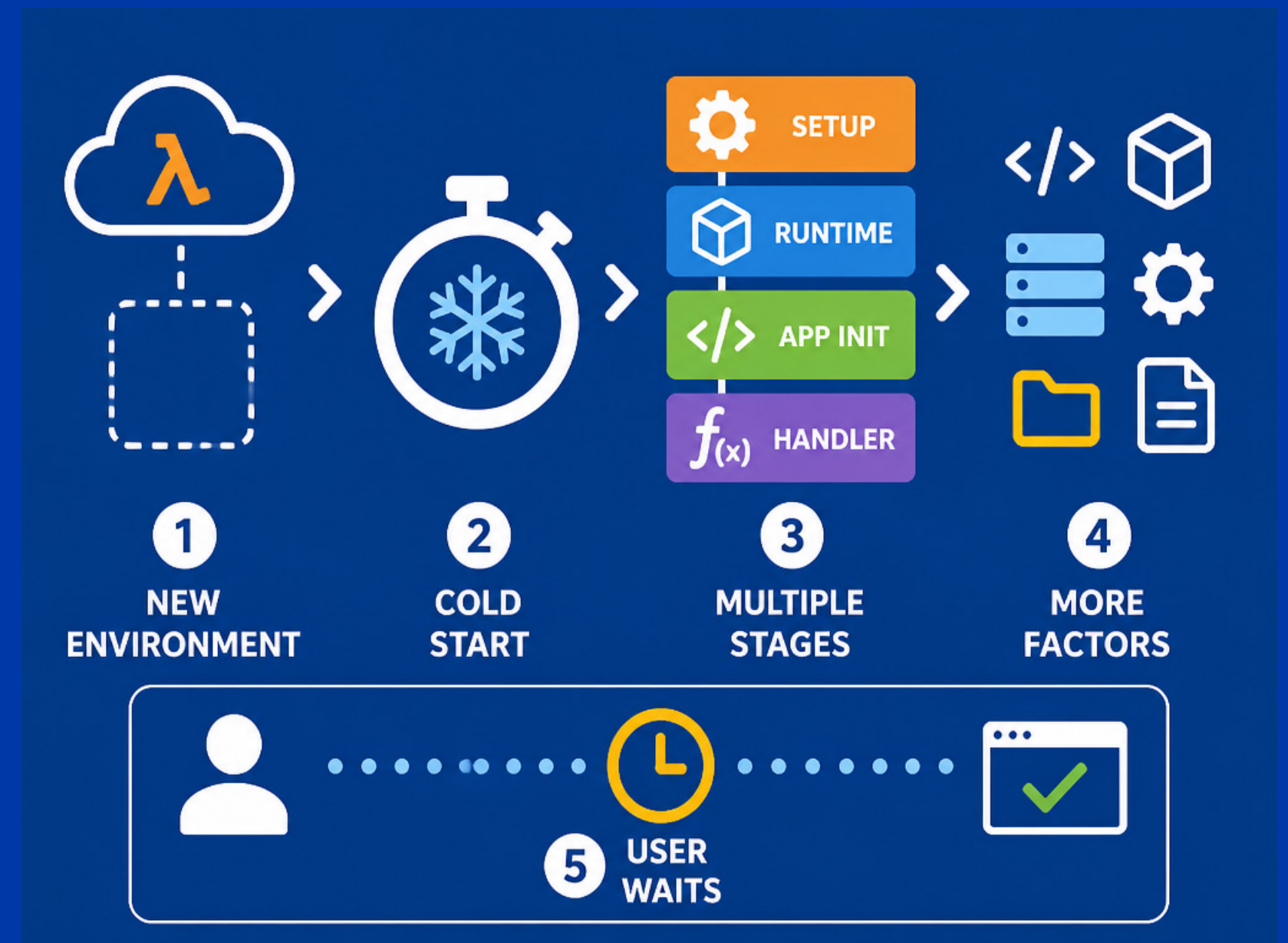
Work happens before your code runs: setup, runtime, app init, handler.

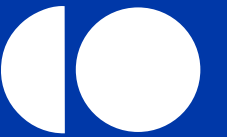
Several factors drive the delay

Language, dependency size, frameworks, config, and init logic add time.

Users feel it as waiting

The extra latency appears before the real business work starts.





Why Java Feels Slower

Heavy work before the first request

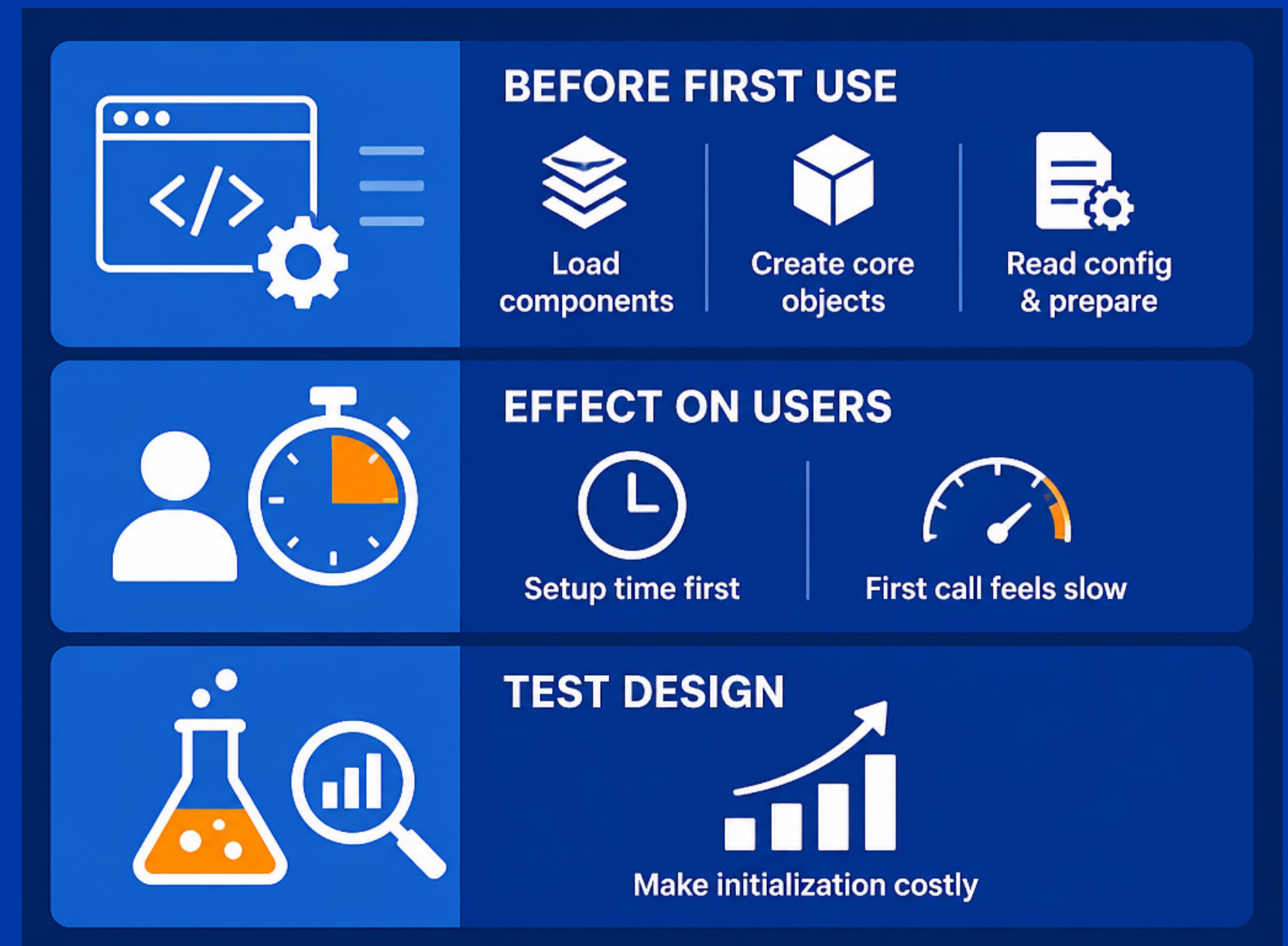
- Java must load many classes and wire up frameworks at startup.
- It builds core objects, reads configuration, and prepares resources early.

Impact on user experience

- That setup time is paid before any real business logic runs.
- The very first request can feel slow, even when code is fast.

Why the test made it obvious

- The experiment intentionally made initialization expensive to expose delays.





SnapStart Overview and Experiment Setup



How SnapStart Helps

Pre-warmed execution state

SnapStart captures a ready-to-run Java environment ahead of real traffic.

Restore instead of re-initialize

On a cold start, Lambda restores the saved snapshot rather than repeating startup work.

Less startup overhead

Expensive initialization happens once, not for each new environment creation.

More predictable latency

Cold-invocation response times become more consistent and easier to plan around.

Best fit use cases

Especially helpful for latency-sensitive Java functions that see cold starts.



**Pre-warmed
environment**



**Restore
on demand**



**One-time
initialization**

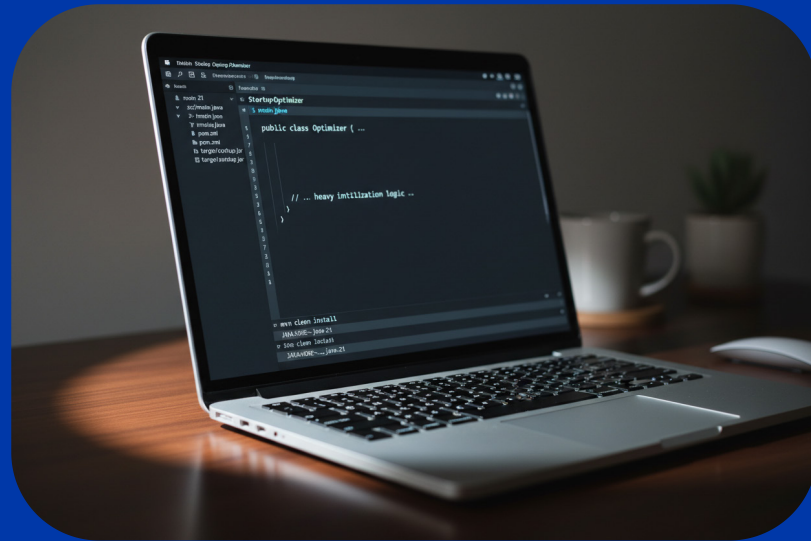


**Consistent
response times**



**Ideal for latency-sensitive
Java functions**

Project Setup



Build and runtime stack

- Used Java 21 and Maven to compile and package the Lambda JAR.
- Designed to stress startup work, not deliver app features.



AWS deployment components

- Deployed the function on AWS Lambda with an IAM execution role.
- Used CloudWatch logging for consistent, repeatable measurement.



What we measured

- Captured Init Duration and Restore Duration to compare startup paths.
- Focused on cold-start behavior to isolate performance impact.

10.



How We Tested

Build the test function

- Create a Java 21 AWS Lambda and add a deliberately slow startup step.

Measure the baseline

- Invoke without SnapStart and record CloudWatch **Init Duration**.

Enable SnapStart correctly

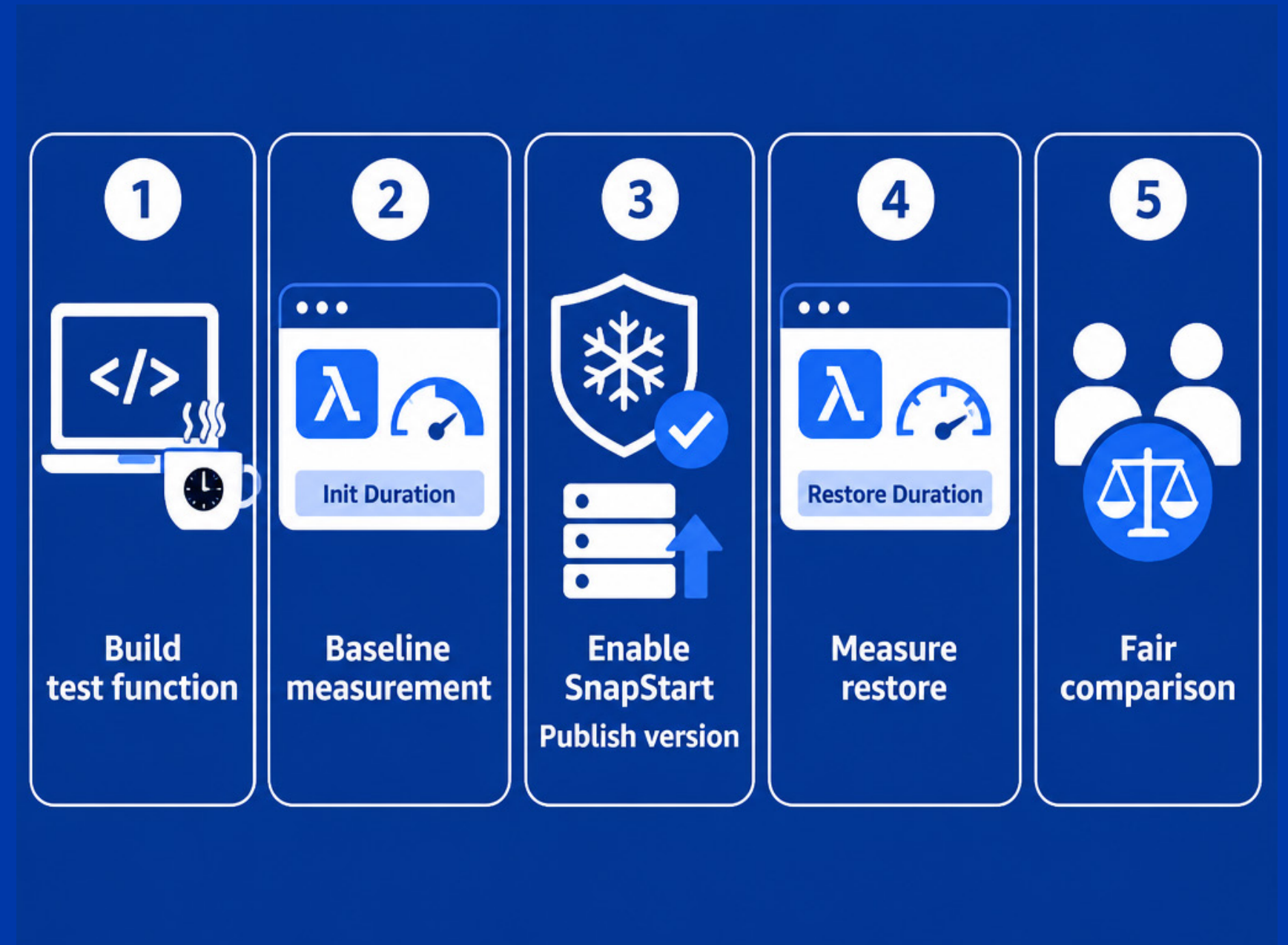
- Turn on SnapStart for **PublishedVersions**, then publish a new version.

Measure the restore path

- Invoke the published version and record CloudWatch **Restore Duration**.

Make it a fair comparison

- Use the same code and settings so the only change is SnapStart.

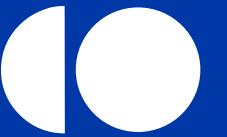


11.



Results, Benefits, and Next Steps

12.



What Changed

Cold-start time comparison

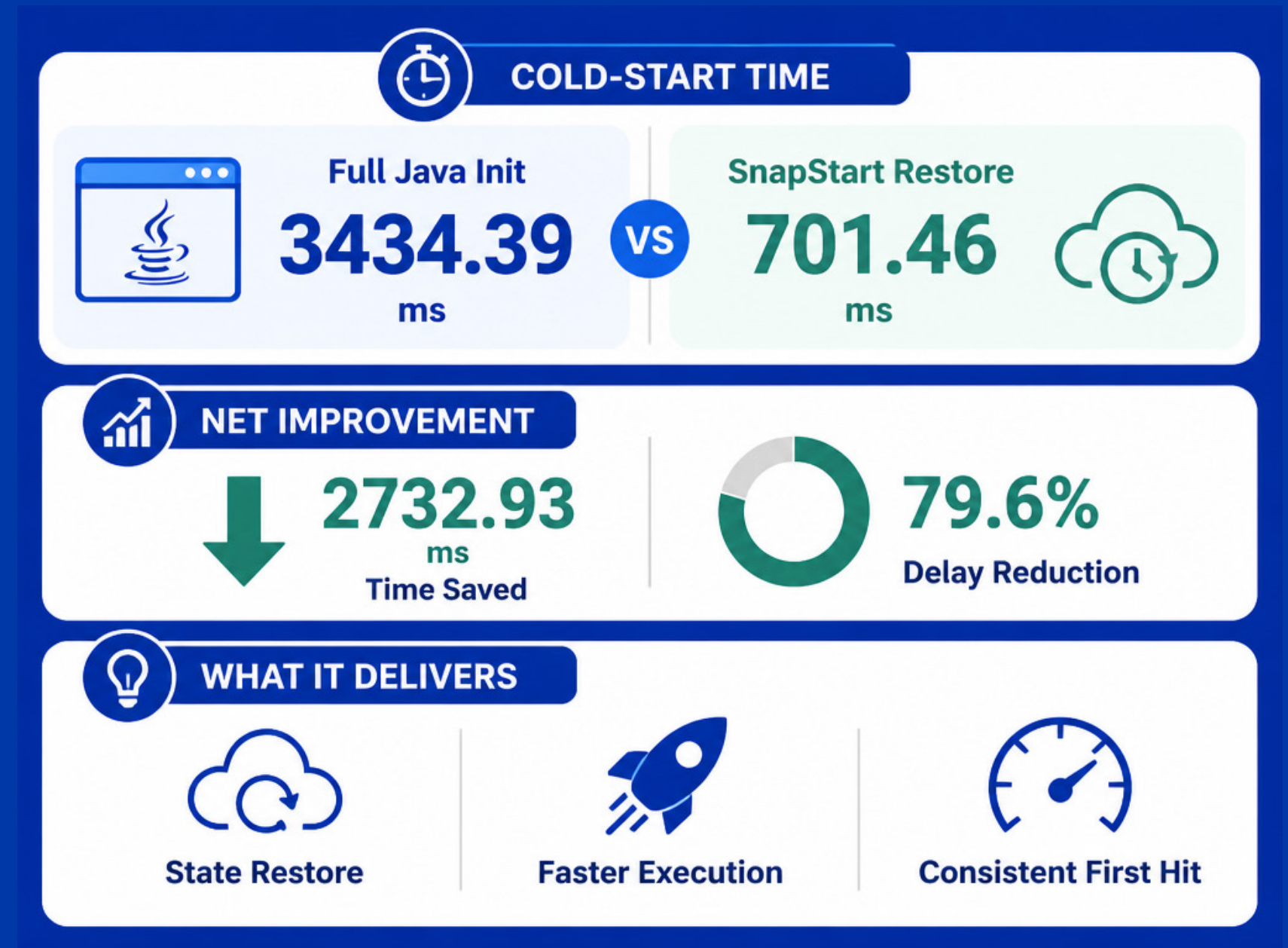
- Full Java initialization took **3434.39 ms** on cold start.
- SnapStart restored the prepared state in **701.46 ms**.

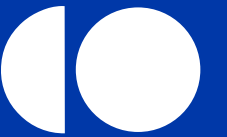
Net improvement

- Startup time dropped by about **2732.93 ms** overall.
- That is roughly a **79.6%** reduction in cold-start delay.

What it means

- Restoration avoids repeating heavy startup work in new environments.
- Users reach business logic faster, with more predictable first-hit latency.





Benefits And Tradeoffs

Where it helps most

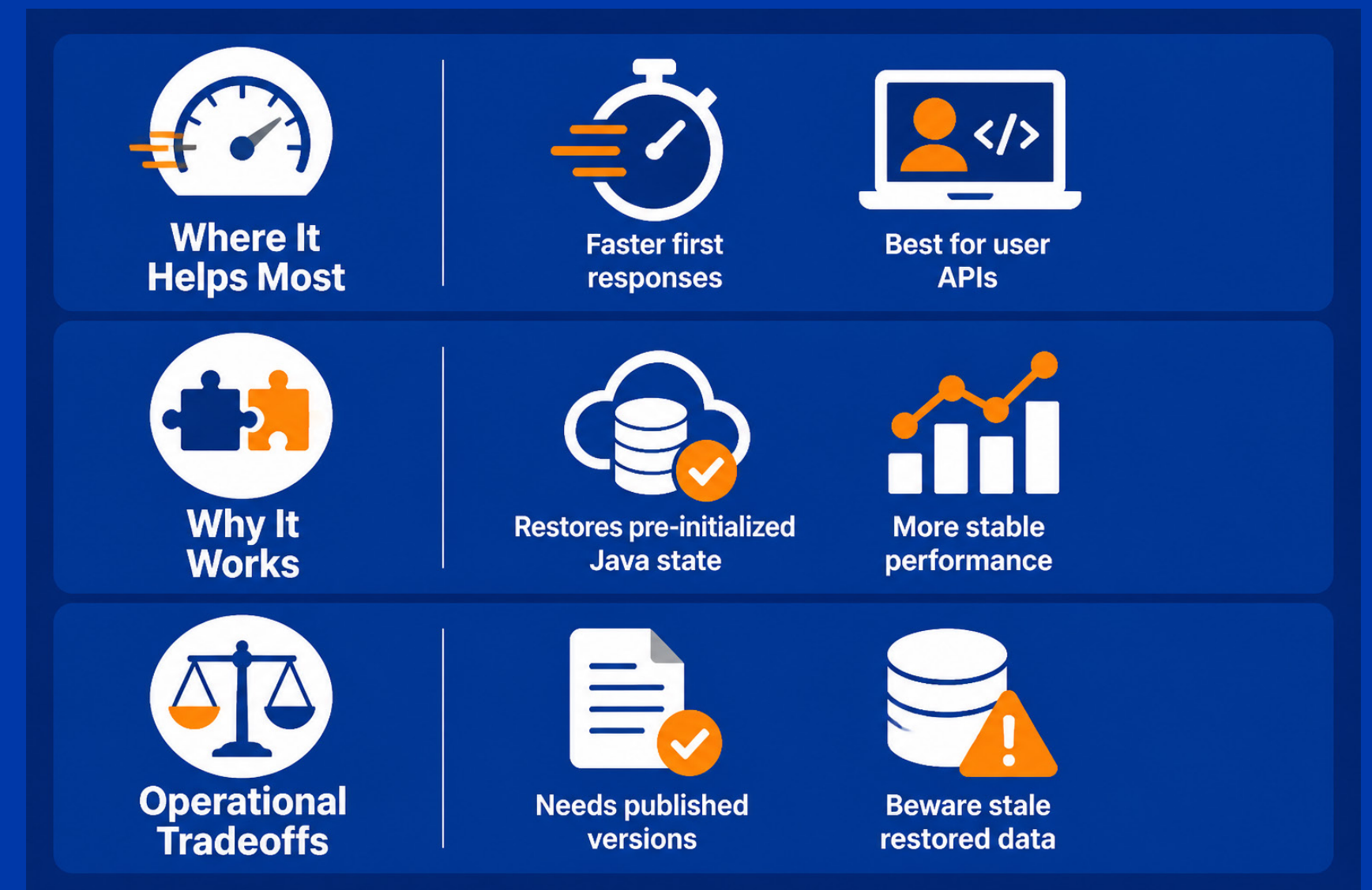
- Delivers faster first responses when new Lambda environments spin up.
- Best fit for user-facing or API flows where delays are obvious.

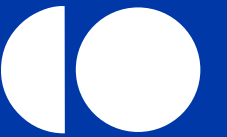
Why it works

- Restores a pre-initialized Java state instead of repeating startup work.
- Keeps performance steadier when cold starts happen unexpectedly.

Operational tradeoffs

- Requires published function versions, so deployments need version discipline.
- Treat restored initialization data carefully to avoid stale state issues.





Where To Go Next

Key takeaway

- SnapStart can significantly cut Java cold-start delay in practice.

What to validate next

- Repeat tests with larger applications and heavier dependency sets.
- Compare different initialization patterns to find main bottlenecks.
- Try a real Spring Boot workload to confirm benefits at scale.

How to decide

- Measure both Init and Restore times, then choose for your needs.
- Balance **latency gains** with how you deploy and version functions.

